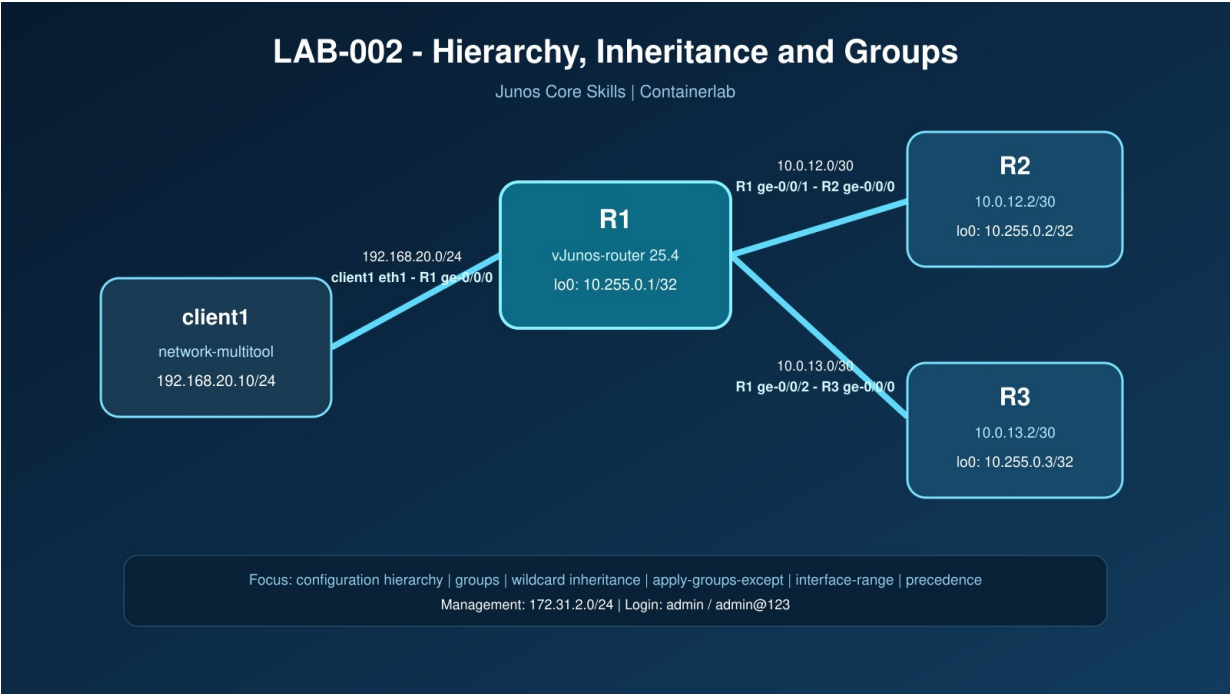


LAB-002 - Task-Only

Junos Core Skills | LAB-002 | Intermediate



Assessment brief

You are standardizing configuration on three Junos PE/P routers. The baseline service is already operational. Your job is to reduce repeated configuration while preserving explicit exceptions and proving the effective configuration. Do not use redeployment to hide a failed change or recovery.

Item	Value
Phase	1 - Junos Core Skills
Difficulty	Intermediate
Estimated time	2 hours
Dependency	LAB-001
Mastery outcome	Build reusable groups, use `apply-groups-except`, build interface ranges, and inspect inherited configuration

Topology and addressing

Node	Interface	Address	Purpose
client1	eth1	192.168.20.10/24	Traffic and reachability test
R1	ge-0/0/0	192.168.20.1/24	Client LAN
R1	ge-0/0/1	10.0.12.1/30	R1-R2
R2	ge-0/0/0	10.0.12.2/30	R2-R1
R1	ge-0/0/2	10.0.13.1/30	R1-R3
R3	ge-0/0/0	10.0.13.2/30	R3-R1
R1/R2/R3	lo0.0	10.255.0.1-3/32	Router identity

Rules of engagement

- Capture pre-checks before each change.
- Use ``show | compare``, ``commit check``, and a meaningful commit comment.
- Prove effective configuration with inheritance display commands, not only the group definition.
- Run data-plane checks after every commit that can affect interfaces.
- Preserve evidence of precedence tests and the final cleanup.
- Do not open the solution guide until finished or blocked.

Task 1 - Deploy and prove the baseline

Deploy the topology from a clean state. Prove that all startup configurations were loaded, all intended interface addresses and static routes exist, and ``scripts/verify.sh`` returns ``RESULT: PASS``.

Required evidence: deployment output, ``show interfaces terse``, exact loopback routes, R1 exact static routes to R2/R3 loopbacks, and verification-script output.

Task 2 - Navigate the Junos hierarchy efficiently

On R1, demonstrate at least two safe methods to move between configuration hierarchy levels. Add a candidate-only system location, inspect it from the relevant hierarchy and from the top, then discard it without affecting active configuration.

Required evidence: hierarchy prompts, candidate diff, proof that active configuration did not change.

Task 3 - Build and apply a reusable system group

On all routers, create a group named ``SP-BASE`` that supplies:

- a login message containing ``AUTHORIZED ACCESS ONLY``;

- NETCONF over SSH;
- the time zone `Asia/Jakarta`.

Apply the group globally. Avoid duplicating these statements as explicit active configuration. Prove both the group definition and the inherited effective configuration.

Required evidence: group definition, effective inherited configuration, comparison, validation, commit history.

Task 4 - Create a router-specific exception

R3 is a controlled exception and must not inherit `SP-BASE`. Use `apply-groups-except`; do not delete the group itself. Prove that R1 and R2 still inherit the group while R3 does not. Then restore inheritance on R3.

Required evidence: exclusion configuration, effective configuration on all three routers, rollback or corrective diff, final verification.

Task 5 - Use a wildcard interface group

On R1, create group `CORE-MTU` using a wildcard interface expression that targets only `ge-0/0/1` and `ge-0/0/2`. The group must supply an interface MTU of 2000. Apply it globally and prove:

- both core-facing interfaces inherit MTU 2000;
- the client-facing `ge-0/0/0` does not inherit it;
- existing addresses and reachability remain intact.

Required evidence: wildcard group definition, inherited views of all three interfaces, interface operational MTU, `RESULT: PASS`.

Task 6 - Apply an interface-level exception

Exclude `CORE-MTU` only from R1 `ge-0/0/2`, leaving `ge-0/0/1` inherited. Prove the difference in effective and operational configuration. Restore the standard after completing the observation.

Required evidence: before/after inheritance output, operational MTU, comparison, final restoration.

Task 7 - Compare explicit and interface-range configuration

On R1, create interface range `CORE-LINKS` containing `ge-0/0/1` and `ge-0/0/2`. Use the range to supply the common description `SP-CORE-MEMBER`.

The startup configuration already provides the explicit descriptions `R1-TO-R2` and `R1-TO-R3`. Do not assume that the range description will replace them. Determine which source wins, then temporarily

remove the explicit description from `ge-0/0/1` and prove that the range description becomes effective. Restore the original description before completing the task.

Required evidence: interface-range definition, inheritance views before and after removing the explicit statement, operational descriptions, your precedence conclusion, final clean configuration.

Task 8 - Prove precedence between overlapping interface ranges

On R1, create two interface ranges named `HOLD-A` and `HOLD-B`. Make `ge-0/0/1` a member of both ranges. Configure `hold-time up 1000` under `HOLD-A` and `hold-time up 2000` under `HOLD-B`.

Before validation, predict whether Junos will reject the overlap or resolve it through inheritance priority. Run `commit check`, commit the configuration, and prove the effective value. Then place `HOLD-B` before `HOLD-A` in configuration order and prove whether the effective value changes. Remove both test ranges when finished.

Do not modify MTU in this task; MTU remains owned by the `CORE-MTU` group from Task 5.

Required evidence: initial range order, successful validation, inheritance output for both orders, your precedence conclusion, cleanup diff, final MTU inheritance, and `RESULT: PASS`.

Completion gate

LAB-002 is complete only when:

- startup configuration loading is proven;
- reusable groups and exceptions work as intended;
- inherited values are distinguished from explicitly configured values;
- explicit member-interface precedence over interface-range expansion is demonstrated;
- baseline forwarding remains healthy;
- overlapping interface-range priority is demonstrated and the test configuration is removed;
- all commits have useful comments and the final `verify.sh` result is PASS.